THE WISDOM CHURCH · ENGINEERING

Google Maps Navigation Setup Guide

Production setup for the integrated “Take me to church” route planner, traffic-aware backend, and secure embedded map.

Audience: Engineering and deployment administrators
Systems: Wisdom Church Go backend and Next.js frontend
Document version: 1.0 · 5 August 2026

1. What this setup activates

The visitor grants location access in the browser. The frontend sends the coordinates through the existing same-origin API proxy to the Wisdom Church backend. The backend uses a private Google Routes API key and the verified church Place ID to calculate a traffic-aware route. The frontend renders the route using a separately restricted Maps JavaScript API key.

> **Privacy:** Visitor coordinates are processed transiently. They must not be stored in a database, application logs, analytics payloads, or error-reporting metadata.

Required values

Variable	Location	Purpose
`GOOGLE_MAPS_ROUTES_API_KEY`	Backend only	Private credential for traffic-aware route calculation.
`GOOGLE_CHURCH_PLACE_ID`	Backend	Verified, unambiguous destination used by Google Routes.
`NEXT_PUBLIC_GOOGLE_MAPS_BROWSER_API_KEY`	Frontend	Domain-restricted key used to render the embedded map.
`NEXT_PUBLIC_CHURCH_GOOGLE_PLACE_ID`	Frontend	Verified destination for the optional native navigation handoff.

2. Create or select the Google Cloud project

1. Open console.cloud.google.com and sign in with the organisation account.
2. Select an existing project or create **Wisdom Church Navigation**.
3. Open **Billing** and attach an active billing account. Google Maps Platform requires billing even when usage remains inside an applicable free allowance.
4. Create a budget and billing alerts before enabling production traffic.

> **Do not use a developer's personal Google Cloud project.** Use an organisation-owned project with at least two administrators and documented recovery access.

3. Enable the correct Google APIs

1. Open **APIs & Services → Library**.
2. Search for **Routes API** and select **Enable**.
3. Search for **Maps JavaScript API** and select **Enable**.

Do not enable unrelated APIs “just in case.” Each credential will be restricted to the single API it needs.

4. Create the private backend Routes key

1. Open **APIs & Services → Credentials**.
2. Select **Create credentials → API key**.
3. Edit the new key and name it **Wisdom Church Routes Backend**.
4. Under **API restrictions**, choose **Restrict key** and select only **Routes API**.
5. If the backend has fixed outbound IP addresses, add them under **Application restrictions → IP addresses**. If the deployment platform has no fixed outbound IP, keep the credential exclusively in its encrypted server environment and retain the Routes-only API restriction.
6. Copy the key into the backend's secret manager. Never send it in chat, commit it, or place it in frontend configuration.

```
GOOGLE_MAPS_ROUTES_API_KEY=replace_with_private_server_key
```

> **Critical:** Never prefix this variable with `NEXT_PUBLIC_`. Doing so would expose the private server key in browser JavaScript.

5. Create the browser Maps key

1. Create a second API key and name it **Wisdom Church Maps Browser**.
2. Under **Application restrictions**, select **Websites**.
3. Add the authorised referrers below, adjusting local ports if required.
4. Under **API restrictions**, select only **Maps JavaScript API**.

```
https://wisdomchurchhq.org/*
https://www.wisdomchurchhq.org/*
http://localhost:3000/*
http://localhost:2000/*
```

```
NEXT_PUBLIC_GOOGLE_MAPS_BROWSER_API_KEY=replace_with_browser_key
```

A browser key is visible by design. Its safety comes from strict HTTP-referrer and API restrictions.

6. Obtain and verify the church Place ID

1. Open Google's Place ID documentation and finder: developers.google.com/maps/documentation/places/web-service/place-id.
2. Search for the exact church listing or the verified Honor Gardens destination.
3. Select the result and copy the Place ID. It commonly begins with `ChIJ`, but the exact format is controlled by Google.
4. Open the result in Google Maps and confirm the entrance, road, venue name, and surrounding landmarks.
5. Have a second team member independently verify it before production deployment.

```
GOOGLE_CHURCH_PLACE_ID=replace_with_verified_place_id
NEXT_PUBLIC_CHURCH_GOOGLE_PLACE_ID=replace_with_the_same_verified_place_id
```

> **No guesswork:** The backend will refuse to calculate routes when the verified Place ID is missing. It does not silently guess from a postal-address string.

7. Configure the backend

Add these values to `Backend-dev/.env` for local development and to the backend deployment platform's encrypted environment for production:

```
NAVIGATION_ENABLED=true
GOOGLE_MAPS_ROUTES_API_KEY=replace_with_private_server_key
GOOGLE_CHURCH_PLACE_ID=replace_with_verified_place_id
NAVIGATION_PROVIDER_TIMEOUT=12s
NAVIGATION_RATE_LIMIT_RPM=10
NAVIGATION_RATE_LIMIT_BURST=3
```

Setting	Meaning
`NAVIGATION_ENABLED`	Explicit feature switch. Set to `false` for an emergency shutdown.
`NAVIGATION_PROVIDER_TIMEOUT`	Maximum time the backend waits for Google before returning a safe error.
`NAVIGATION_RATE_LIMIT_RPM`	Maximum route calculations per visitor IP during each minute.
`NAVIGATION_RATE_LIMIT_BURST`	Short burst allowance before throttling begins.

8. Configure the frontend

Add these values to `Frontend-dev/.env.local` and the frontend hosting environment:

```
NEXT_PUBLIC_GOOGLE_MAPS_BROWSER_API_KEY=replace_with_browser_key
NEXT_PUBLIC_CHURCH_GOOGLE_PLACE_ID=replace_with_verified_place_id
```

The frontend continues using its existing backend proxy configuration. Ensure the production backend origin remains correct:

```
API_PROXY_TARGET=https://api.wisdomchurchhq.org
```

9. Local validation

Backend

```
cd Backend-dev
make test
go vet ./...
make run
```

Frontend

```
cd Frontend-dev
make check
make dev
```

1. Open the local website over `localhost`.
2. Select **Take me to church**.
3. Allow location access.
4. Confirm the embedded map remains inside the website.
5. Verify the displayed destination, route line, ETA, and distance.
6. Test permission denial and confirm the fallback message is understandable.

10. Production deployment order

1. Configure backend secrets and keep `NAVIGATION_ENABLED=false`.
2. Deploy the backend and confirm `/healthz` and `/readyz`.
3. Configure and deploy the frontend browser key and Place ID.
4. Set `NAVIGATION_ENABLED=true` and redeploy or restart the backend.
5. Run a real route test from a safe location and verify the exact church destination.
6. Review Google Cloud metrics, backend latency, errors, and rate-limit responses.

11. Troubleshooting

Symptom	Likely cause	Resolution
“For development purposes only” watermark	Billing, key, or Maps JavaScript API configuration	Confirm billing, enable the API, and check browser-key restrictions.
Map does not load	Current domain is not an authorised referrer	Add the exact HTTPS domain pattern to the browser key.
Route preview returns 503	Feature disabled, missing server key/Place ID, or provider unavailable	Check backend configuration and server logs without logging coordinates.
Route preview returns 429	Per-IP limit reached	Wait for the indicated retry period; investigate unusual repeated traffic.
Wrong destination	Incorrect Place ID	Disable navigation immediately and re-verify the Place ID with two people.
Location permission denied	User or browser privacy setting	Explain how to enable permission; never attempt to bypass the browser decision.

12. Cost and security controls

- ☐ Billing budget and alert recipients configured.
- ☐ Routes key restricted to Routes API.

- [ ] Browser key restricted to Maps JavaScript API and approved referrers.
- [ ] Redis is available in production for distributed rate limiting.
- [ ] Coordinates are absent from logs, analytics, traces, and databases.
- [ ] Google Cloud quotas are reviewed and set to an acceptable ceiling.
- [ ] Production Place ID has been independently verified.
- [ ] Emergency disable procedure is documented: `NAVIGATION_ENABLED=false`.

13. Final activation checklist

- [ ] Organisation-owned Google Cloud project selected.
- [ ] Billing and budget alerts enabled.
- [ ] Routes API enabled.
- [ ] Maps JavaScript API enabled.
- [ ] Private backend key created and restricted.
- [ ] Browser key created and domain-restricted.
- [ ] Verified Place ID configured in both deployment environments.
- [ ] Backend and frontend validation commands pass.
- [ ] Permission-granted and permission-denied flows tested.
- [ ] Production route reaches the exact church entrance.

Official references: Routes API setup · Maps JavaScript API setup · Google Maps Platform pricing